STUDENT AI-READINESS HANDOUT & AUDIT TRACKER Part 3 • Printable 4-Page Practical Workbook

Name of Student:	_____ ___ _____	**Degree / Branch:**	_____
College:	_____ ___ _____	**Academic Year:**	_____

Instructions for Students & Faculty: This 4-page practical workbook is designed to be kept throughout your engineering degree. Use it to audit your coursework, grade your personal projects, track your daily commits, and prepare for technical interviews across semesters 2-1 through 4-2.

Module 1: The Coursework-to-AI Translation Checklist

Check off each capability as you master the connection between academic theory and industry engineering:

Year 2: Semesters 2-1 & 2-2 (Foundations & Core Ingestion)

- ■ Linear Algebra & Vector Spaces: I can explain how matrix multiplication, dot products, and projections translate into vector search and model embeddings.
- ■ Algorithmic Complexity: I can calculate Big-O time and space complexity (O(1), O(log n), O(n), O(n log n), O(n^2)) on whiteboards without guessing.
- ■ Memory & Pointer Mechanics: I understand how data is allocated in heap vs. stack, how references operate, and how memory leaks occur.
- ■ Relational Data & Indexing: I can write multi-table joins, subqueries, and window functions in SQL, and I understand why B-Tree indexing prevents table scans.
- ■ Git Discipline: I use Git from the command line daily, writing clean commit messages and maintaining organized branches.

Year 3: Semesters 3-1 & 3-2 (Systems, AI Augmentation & Feature Engineering)

- ■ Context & Boundary Direction: When using AI coding assistants, I provide structured type definitions, edge-case constraints, and architectural context.
- ■ Hallucination & Logic Auditing: I actively review AI-generated code for silent bugs, injection vulnerabilities, and performance bottlenecks before running it.
- ■ Networking & API Protocols: I understand RESTful architecture, HTTP status codes (200, 400, 404, 500), JSON serialization, and latency budgets.
- ■ Applied Evaluation Metrics: I can calculate and explain Precision, Recall, F1-Score, and ROC-AUC, and I know why accuracy fails on imbalanced datasets.
- ■ Sovereign Project Construction: I have built an end-to-end application connecting a backend API, a database, and a frontend interface without tutorial walkthroughs.

Year 4: Semesters 4-1 & 4-2 (Production, Scaling & Deployment)

- ■ Containerization: I can write a Dockerfile, build a minimal image, and run my service inside a container locally and in the cloud. ■ Public Deployment: My project is accessible via a live public URL with interactive API documentation (e.g., Swagger / OpenAPI). ■ Defensive Engineering: My code features structured exception logging, input validation schemas, and graceful error responses.
- ■ Business Impact Framing: I can explain my project in 60 seconds focusing on the problem solved, architectural trade-offs, and quantifiable metrics.

Module 2: The Project-to-Proof 20-Point Scoring Rubric

Evaluate your primary technical resume project against the rubric below. Aim for a score of 16 or higher before listing it on your placement resume.

Evaluation Dimension	Level 1: Tutorial Clone (1 Pt)	Level 3: Working Prototype (3 Pts)	Level 5: Production-Ready Proof (5 Pts)	Your Score
1. Problem Originality	Downloaded a clean Kaggle dataset (Iris, Titanic, Housing).	Addressed a known problem using freshly scraped or assembled data.	Solved an actual, messy community, campus, or business problem.	_____ / 5
2. Deployment & Access	Runs only in a local Jupyter Notebook or local terminal window.	Deployed on localhost with a basic Streamlit or Flask interface.	Deployed to a public cloud URL with a documented, live REST API.	_____ / 5
3. Code Rigor & Testing	Single .py or .ipynb file; zero tests; no dependencies specified.	Modular code with a requirements.txt and basic run instructions.	Containerized (Dockerfile), unit tests included, clean input validation.	_____ / 5
4. Architectural Depth	Default README or empty GitHub repo description.	Basic markdown README explaining how to run the project locally.	Architecture diagram, benchmark latency stats, and trade-off whitepaper.	_____ / 5

Total Score: _____ / 20

16–20 Points (Production-Ready)	Strong engineering proof. Highlights deep capability during technical rounds.
11–15 Points (Competent Prototype)	Good foundation. Needs containerization, public hosting, and automated testing.
0–10 Points (Tutorial Tier)	Common classroom assignment. Banish from your placement resume and re-architect.

Module 3: Resume Bullet Transformation Formula

Transform generic academic claims into high-impact engineering specifications using the **XYZ Formula:** Accomplished [X], as measured by [Y], by doing [Z].

■ BEFORE (Passive Student Claim)	■ AFTER (Engineering Specification)
"Built a machine learning project to predict customer churn using Python and Pandas."	"Architected an end-to-end churn classification service on 100K customer records using XGBoost; deployed as a containerized REST API with sub-30ms latency, achieving an 89% F1-score and reducing false-positive customer alerts by 24%."
"Created a database management system project for a college library in SQL."	"Designed and indexed a PostgreSQL relational schema managing 50K book records; built analytical window queries that cut search latency by 65% across concurrent sessions."

Module 3: Resume Bullet Transformation — Your Turn

Use the XYZ formula to rewrite one of your own project bullets. Focus on measurable evidence rather than listing tools alone.

My Original Academic Bullet
_____
_____
My Engineering Specification
_____
_____
_____

Daily Git Commit & Evidence Tracker

Use this mini-log to build a visible record of consistent engineering practice.

Date	What I Built / Changed	Commit / Evidence	Reviewed / Tested?

Module 4: The AI-Era Interview Defense Sheet

Before entering a technical interview, write out concise, technical answers to these four defense questions for your primary project.

1. The Architectural Trade-Off:

"Why did you choose your specific database and model architecture over common alternatives, and where does this design fail?" **My Answer:**

_____

_____

_ _____

_____

_ _____

_____

_ _____

2. The Failure Mode & Edge Case:

"What is the worst-case time complexity of your data pipeline, and what happens when invalid or corrupted input hits your API?" **My Answer:**

_____

_____

_ _____

_____
_ _____

_____
_ _____

3. The AI Verification Process:

"If an AI assistant helped write any part of this codebase, what specific logic bugs or performance bottlenecks did you catch during code review?"

My Answer:

_____

_____
_ _____

_____
_ _____

_____
_ _____

4. The Quantified Business Impact:

"If this application were deployed in a commercial environment tomorrow, what specific operational metric, latency figure, or cost would it improve?"

My Answer: _____

_____
_ _____

_____
_ _____

_____
_ _____

Student self-audit reminder: Your goal is not merely to use AI tools. Your goal is to understand, verify, deploy, measure, and defend the engineering decisions behind what you build.